

Jalon3 : Modélisation de la base de données

1-Introduction à la méthode MERISE

Dans le cadre du développement de la plateforme MENTOR, la conception de la base de données a été réalisée en suivant la méthode MERISE, une méthode d'analyse et de conception des systèmes d'information largement utilisée pour modéliser les données de manière structurée.

L'objectif principal de MERISE est de concevoir une base de données fiable, cohérente et évolutive, tout en séparant les besoins métier des contraintes techniques liées au système de gestion de base de données (SGBD). Cette approche facilite l'analyse des besoins, limite les redondances de données et garantit l'intégrité des informations tout au long du développement de l'application.

La démarche MERISE repose sur trois niveaux de modélisation complémentaires :

- **Le Modèle Conceptuel de Données (MCD)** : il représente les entités métier, leurs attributs ainsi que les relations qui les unissent, indépendamment de toute contrainte technique.
- **Le Modèle Logique de Données (MLD)** : il traduit le MCD en un modèle relationnel composé de tables, de clés primaires et de clés étrangères, adapté au modèle relationnel.
- **Le Modèle Physique de Données (MPD)** : il correspond à l'implémentation concrète de la base de données dans le SGBD choisi. Il définit les types SQL, les contraintes d'intégrité, les index ainsi que la structure des tables.

Pour le projet **MENTOR**, cette démarche a permis de concevoir une base de données répondant aux besoins fonctionnels de la plateforme de réservation de formations. Le modèle obtenu facilite la gestion des utilisateurs, des formateurs, des formations, des disponibilités, des réservations et des paiements, tout en garantissant la cohérence et la fiabilité des données.

Les sections suivantes présentent successivement le dictionnaire des données, le Modèle Conceptuel de Données (MCD), le Modèle Logique de Données (MLD), le Modèle Physique de Données (MPD), ainsi que la justification des choix de modélisation et la vérification de la normalisation de la base de données.

2- Dictionnaire de donnée

Code	Signification	Type	Taille	Nature	Observation
Id_utilisateur	Identifiant de l'utilisateur	AN	100	E	Identifiant
Nom_Utilisateur	Nom de l'utilisateur	AN	100	E	
Prenom_Utilisateur	Prénom de l'utilisateur	AN	100	E	
Email_utilisateur	Email de l'utilisateur	AN	100	E	
Mot_de_passe	Mot de passe chiffré	AN	100	E	
Role_utilisateur	Rôle utilisateur	AN	100	E	
Photo_utilisateur	Photo de profil	AN	255	E	URL de l'image
Date_creation	Date d'inscription	DATE	100	E	JJ/MM/AAAA :H-m-s
Id_Formateur	Identifiant du formateur	N	11	E	Identifiant
Expérience	Expérience professionnelle du formateur	AN	100	E	
Vidéo_Présentation	Vidéo de présentation du formateur	AN	255	E	URL de la vidéo
Langue	Langue parlée par le formateur	AN	100	E	
Localisation	Localisation du formateur	AN	100	E	
Id_Formation	Identifiant de la formation	N	11	E	Identifiant
Titre	Titre de la formation	AN	105	E	
Description	Description de la formation	AN	1000	E	

Durée	Durée de la formation	AN	50	E	
Prix	Prix de la formation	DECIMAL	10,2	E	Euro
Image	Image de la formation	AN	255	E	URL
Id_disponibilité	Identifiant de la disponibilité	N	11	E	Identifiant
Date	Date de la disponibilité	DATE		E	
Heure_début	Heure de début	HEURE		E	
Heure_fin	Heure de fin	HEURE		E	
Statut	Le statut de la disponibilité	AN		E	
Id_paiement	Identifiant du paiement	N	11	E	Identifiant
Montant	Montant du paiement	DECIMAL	10,2	E	
Mode	Mode du paiement	AN	50	E	
Statut	Statut du paiement	AN	20	E	Payé, En attente

Légende

Type

- A : Alphabétique
- N : Numérique
- AN : Alphanumérique
- DATE : Date
- HEURE : Heure
- DECIMAL : Nombre décimal

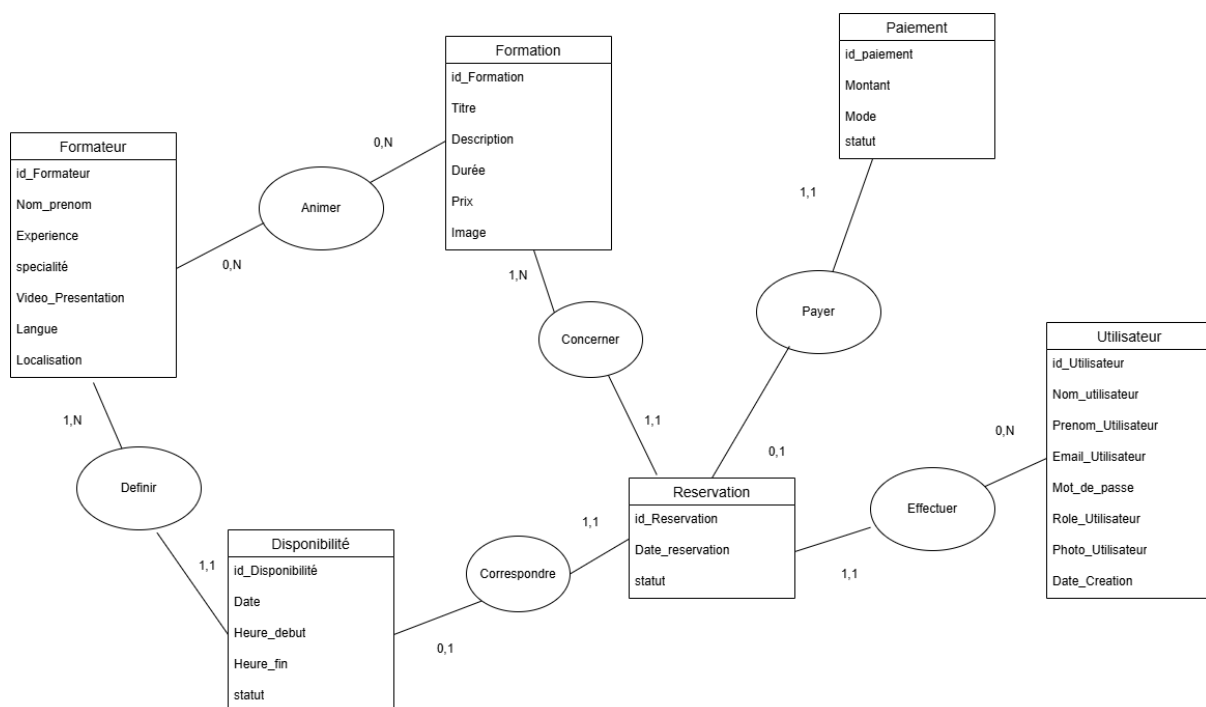
Nature

- E : Élément obligatoire
- F : Élément facultatif

Observation

- Identifiant : clé primaire

3-Modele conceptuel de donnée (MCD)



4-Modele Logique de donnée (MLD)

1. Définition

Le Modèle Logique des Données (MLD) est une représentation des données adaptée au modèle relationnel. Il est obtenu à partir du Modèle Conceptuel des Données (MCD) en appliquant les règles de passage de la méthode Merise.

Le MLD permet de transformer les entités et les associations du MCD en tables relationnelles. Il précise les clés primaires ainsi que les clés étrangères qui assurent les relations entre les différentes tables de la base de données.

2. Règles de passage du MCD au MLD

Règle 1 : Cas des entités

Chaque entité devient une table.

L'identifiant de l'entité devient la clé primaire de la table.

Les propriétés deviennent les attributs de cette table.

Règle 2 : Cas des relations (1,1) vers (0,N)

L'association disparaît.

La clé primaire de la table située du côté (1,N) migre dans la table située du côté (1,1).

Cette clé devient une clé étrangère précédée du symbole #.

Règle 3 : Cas des relations (N,N)

Une nouvelle table est créée.

Sa clé primaire est composée de la concaténation des clés primaires des deux tables reliées.

Les éventuels attributs de l'association deviennent des colonnes de cette nouvelle table.

3. Modèle Logique des Données

UTILISATEUR (id_utilisateur, nom_Utilisateur, prenom_Utilisateur, email_Utilisateur, mot_de_passe, photo_Utilisateur, role_Utilisateur, date_creation)

FORMATEUR (id_formateur, nom_prenom, experience, spécialité, video_presentation, photo, langue, localisation, #id_utilisateur)

FORMATION (id_formation, titre, description, prix, durée, image, #id_formateur)

ANIMER (#id_formateur, #id_formation)

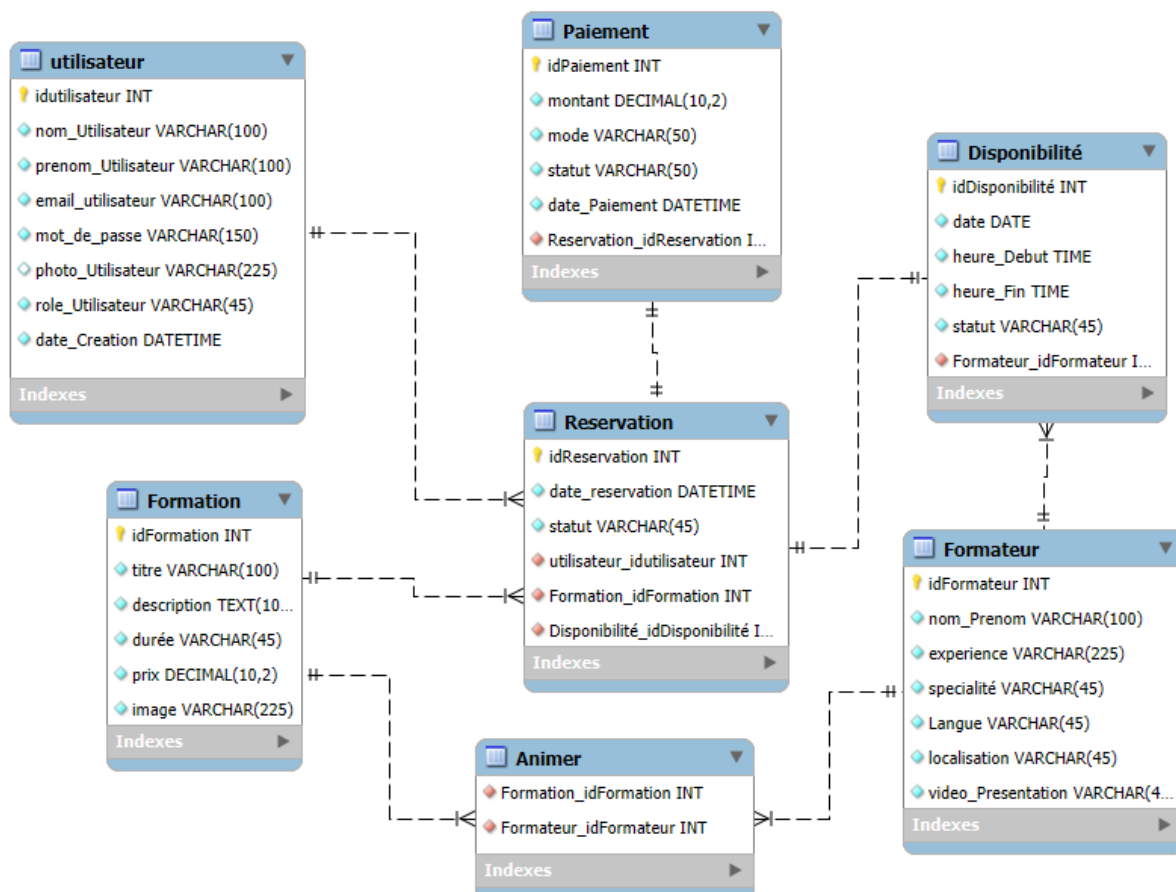
DISPONIBILITE (id_disponibilite, date, heure_debut, heure_fin, statut, #id_formateur)

RESERVATION (id_reservation, date_reservation, statut, #id_utilisateur, #id_formation, #id_disponibilite)

PAIEMENT (id_paiement, montant, mode, statut, #id_reservation)

5- Modele Physique de donnée (MPD)

5-1- Schema du MPD



5-2- Script SQL de la création de la base de données

- utilisateur

```
CREATE TABLE IF NOT EXISTS utilisateur (  
  idutilisateur INT NOT NULL AUTO_INCREMENT,  
  nom_Utilisateur VARCHAR(100) NOT NULL,  
  prenom_Utilisateur VARCHAR(100) NOT NULL,  
  email_utilisateur VARCHAR(100) NOT NULL,  
  mot_de_passe VARCHAR(150) NOT NULL,  
  photo_Utilisateur VARCHAR(225) NULL,  
  role_Utilisateur VARCHAR(45) NOT NULL,  
  date_Creation DATETIME NOT NULL,  
  PRIMARY KEY (idutilisateur),  
  UNIQUE INDEX email_utilisateur_UNIQUE (email_utilisateur ASC) VISIBLE)  
ENGINE = InnoDB;
```

-Table Formateur

```
CREATE TABLE IF NOT EXISTS Formateur (  
  idFormateur INT NOT NULL AUTO_INCREMENT,  
  nom_Prenom VARCHAR(100) NOT NULL,  
  experience VARCHAR(225) NOT NULL,  
  spécialité VARCHAR(45) NOT NULL,  
  Langue VARCHAR(45) NOT NULL,
```

```
localisation VARCHAR(45) NOT NULL,  
video_Presentation VARCHAR(45) NOT NULL,  
PRIMARY KEY (idFormateur)) ENGINE = InnoDB;
```

- Table Formation

```
CREATE TABLE IF NOT EXISTS Formation (  
idFormation INT NOT NULL AUTO_INCREMENT,  
titre VARCHAR(100) NOT NULL,  
description TEXT(100) NOT NULL,  
durée VARCHAR(45) NOT NULL,  
prix DECIMAL(10,2) NOT NULL,  
image VARCHAR(225) NOT NULL,  
PRIMARY KEY (idFormation)) ENGINE = InnoDB;
```

- Table Disponibilité

```
CREATE TABLE IF NOT EXISTS Disponibilité (  
idDisponibilité INT NOT NULL AUTO_INCREMENT,  
date DATE NOT NULL,  
heure_Debut TIME NOT NULL,  
heure_Fin TIME NOT NULL,  
statut VARCHAR(45) NOT NULL,  
Formateur_idFormateur INT NOT NULL,  
PRIMARY KEY (idDisponibilité),
```



```
INDEX fk_Disponibilité_Formateur1_idx (Formateur_idFormateur ASC)  
VISIBLE,
```

```
CONSTRAINT fk_Disponibilité_Formateur1
```

```
FOREIGN KEY (Formateur_idFormateur)
```

```
REFERENCES Formateur (idFormateur)
```

```
ON DELETE NO ACTION ON UPDATE NO ACTION) ENGINE = InnoDB;
```

-Table Reservation

```
CREATE TABLE IF NOT EXISTS Reservation (
```

```
idReservation INT NOT NULL AUTO_INCREMENT,
```

```
date_reservation DATETIME NOT NULL,
```

```
statut VARCHAR(45) NOT NULL,
```

```
utilisateur_idutilisateur INT NOT NULL,
```

```
Formation_idFormation INT NOT NULL,
```

```
Disponibilité_idDisponibilité INT NOT NULL,
```

```
PRIMARY KEY (idReservation),
```

```
INDEX fk_Reservation_utilisateur_idx (utilisateur_idutilisateur ASC)  
VISIBLE,
```

```
INDEX fk_Reservation_Formation1_idx (Formation_idFormation ASC) VISIBLE,  
INDEX fk_Reservation_Disponibilité1_idx (Disponibilité_idDisponibilité  
ASC) VISIBLE,
```

```
CONSTRAINT fk_Reservation_utilisateur
```

```
FOREIGN KEY (utilisateur_idutilisateur)
```

```
REFERENCES mydb.utilisateur (idutilisateur)
```

```
ON DELETE NO ACTION ON UPDATE NO ACTION,
```

```
CONSTRAINT fk_Reservation_Formation1
```

```
FOREIGN KEY (Formation_idFormation)
REFERENCES Formation (idFormation)
ON DELETE NO ACTION ON UPDATE NO ACTION,
CONSTRAINT fk_Reservation_Disponibilité1 FOREIGN KEY
(Disponibilité_idDisponibilité)
REFERENCES Disponibilité (idDisponibilité)
ON DELETE NO ACTION ON UPDATE NO ACTION) ENGINE = InnoDB;
```

- Table Paiement

```
CREATE TABLE IF NOT EXISTS Paiement (
idPaiement INT NOT NULL AUTO_INCREMENT,
montant DECIMAL(10,2) NOT NULL,
mode VARCHAR(50) NOT NULL,
statut VARCHAR(50) NOT NULL,
date_Paiement DATETIME NOT NULL,
Reservation_idReservation INT NOT NULL,
PRIMARY KEY (idPaiement),
INDEX fk_Paiement_Reservation1_idx (Reservation_idReservation ASC)
VISIBLE,
CONSTRAINT fk_Paiement_Reservation1 FOREIGN KEY
(Reservation_idReservation)
REFERENCES mydb.Reservation (idReservation)
ON DELETE NO ACTION ON UPDATE NO ACTION) ENGINE = InnoDB;
```

- Table Animer

```

CREATE TABLE IF NOT EXISTS Animer (
    Formation_idFormation INT NOT NULL,
    Formateur_idFormateur INT NOT NULL,
    INDEX fk_Animer_Formateur1_idx (Formateur_idFormateur ASC) VISIBLE,
    CONSTRAINT fk_Animer_Formation1 FOREIGN KEY (Formation_idFormation)
    REFERENCES Formation (idFormation)
    ON DELETE NO ACTION ON UPDATE NO ACTION,
    CONSTRAINT fk_Animer_Formateur1
    FOREIGN KEY (Formateur_idFormateur)
    REFERENCES Formateur (idFormateur)
    ON DELETE NO ACTION ON UPDATE NO ACTION) ENGINE = InnoDB;

```

6. Justification des choix de modélisation

La modélisation de la base de données de la plateforme MENTOR a été réalisée en appliquant les principes de la méthode MERISE, afin de garantir une organisation cohérente, évolutive et performante des données.

Les entités ont été définies à partir des besoins fonctionnels identifiés dans le cahier des charges. Chaque entité représente un objet métier essentiel au fonctionnement de l'application.

L'entité **Utilisateur** permet de gérer les comptes des apprenants qui utilisent la plateforme. Les informations enregistrées comprennent notamment l'identité, les informations de connexion, le rôle et la photo de profil.

L'entité **Formateur** regroupe les informations relatives aux intervenants, telles que leur expérience professionnelle, leur spécialité, les langues parlées, leur localisation ainsi que leur vidéo de présentation. Ces informations permettent aux utilisateurs de sélectionner un formateur en fonction de leurs besoins.

L'entité **Formation** contient l'ensemble des informations relatives aux formations proposées, notamment le titre, la description, la durée, le prix et l'image de présentation.

La relation **Animer** a été créée afin de représenter la relation plusieurs-à-plusieurs entre les formateurs et les formations. En effet, un formateur peut animer plusieurs formations et une même formation peut être assurée par plusieurs formateurs. Cette relation est matérialisée par une table d'association possédant une clé primaire composite.

L'entité **Disponibilité** permet d'enregistrer les créneaux horaires proposés par chaque formateur. Cette séparation facilite la gestion des plannings et évite les conflits de réservation.

L'entité **Réservation** constitue le cœur du système. Elle relie un utilisateur, une formation et une disponibilité afin de matérialiser l'inscription à une session de formation. Cette organisation permet de conserver un historique complet des réservations.

Enfin, l'entité **Paielement** permet d'assurer le suivi financier des réservations. Chaque paiement est associé à une réservation unique, ce qui garantit la traçabilité des transactions.

Cette modélisation présente plusieurs avantages :

- réduction des redondances de données ;
- meilleure cohérence entre les informations stockées ;
- facilité de maintenance et d'évolution de la base de données ;
- respect de l'intégrité référentielle grâce aux clés primaires et étrangères ;
- optimisation des performances lors des recherches et des traitements.

Ainsi, la structure retenue répond parfaitement aux besoins fonctionnels de la plateforme tout en restant facilement extensible pour intégrer de nouvelles fonctionnalités (avis utilisateurs, certificats, notifications, etc.).